

HOMEPEdia

Senior-Level Implementation Plan

A DataOps + Big Data + Geospatial + AI platform for exploring and analysing the French housing market.

Project basis: the supplied HOMEPEdia specification requires data gathering, relational and non-relational databases, Spark/Hadoop-based Big Data processing, textual analysis, interactive visualisations, and analysis at city, department and region levels.

1. Target Architecture

External Sources → Data Ingestion → Data Lake (RAW/BRONZE/SILVER/GOLD) → Spark/Big Data Processing → PostgreSQL/PostGIS + MongoDB → Analytics Layer → Streamlit Application.

Core components:

- Public/open data sources such as INSEE, data.gouv.fr, DVF, DPE, environmental and infrastructure datasets.
- Python ingestion and validation pipelines.
- Parquet-based data lake.
- Apache Spark / PySpark for distributed processing.
- PostgreSQL + PostGIS for structured and geospatial data.
- MongoDB for semi-structured and textual data.
- Analytics layer for housing, demographics, economy, environment and NLP.
- Streamlit for the interactive application.

2. Define the Product Before Coding

Turn the specification into concrete analytical use cases.

Housing market:

- Price per m²
- Median and average prices
- Transaction volume
- Price evolution
- Property-type distribution

Territorial comparison:

- Housing
- Population
- Income
- Employment
- Education
- Transport
- Energy
- Environment
- Infrastructure

Investment/risk analysis:

Combine housing, demographic, economic, energy, infrastructure and environmental indicators.

Text intelligence:

Process textual data with NLP to extract sentiment, topics, entities and territory-level indicators.

3. Data-Source Strategy

Target 12–50 sources, prioritising open/public datasets where possible.

Recommended domains:

- Housing: DVF
- Population and demographics: INSEE
- Income and employment: INSEE
- Energy: DPE
- Environment and risk: Géorisques / open environmental datasets
- Education: Education nationale
- Transport and infrastructure: open transport datasets
- Geography: official administrative boundaries
- Text: a review or other textual dataset

Create docs/data_sources.md containing:

source name, provider, URL, licence, update frequency, geographic level, temporal coverage, format, ingestion method and reliability.

4. Geographic Data Model

Use canonical geographic identifiers rather than joining on names.

```
Hierarchy:
France
  ■■■ Region
  ■■■ Department
  ■■■ Commune
```

Canonical fields:

- region_code
- department_code
- commune_code
- latitude / longitude
- geometry

Use official geographic codes whenever possible.

5. Repository Structure

```
homepedia/
  ■■■ README.md
  ■■■ Makefile
  ■■■ docker-compose.yml
  ■■■ .env.example
  ■■■ apps/streamlit/
  ■■■ ingestion/
  ■ ■■■ sources/
  ■ ■■■ connectors/
  ■ ■■■ pipelines/
  ■■■ processing/
  ■ ■■■ spark/
  ■ ■■■ cleaning/
  ■ ■■■ transformations/
  ■ ■■■ feature_engineering/
  ■■■ analytics/
  ■ ■■■ housing/
  ■ ■■■ demographics/
  ■ ■■■ geography/
  ■ ■■■ environment/
  ■ ■■■ text/
  ■■■ database/
  ■ ■■■ migrations/
  ■ ■■■ schemas/
  ■ ■■■ seeds/
  ■■■ ml/
  ■ ■■■ nlp/
  ■ ■■■ models/
  ■ ■■■ evaluation/
  ■■■ tests/
  ■■■ notebooks/
  ■■■ docs/
  ■■■ infrastructure/
```

6. Infrastructure

Start with Docker for reproducibility.

Initial services:

- PostgreSQL + PostGIS
- MongoDB

- Spark master
- Spark worker
- Jupyter
- Streamlit

Keep infrastructure reproducible locally. If available, Spark workloads can later be moved to a managed platform such as Databricks.

7. Relational Database

Use PostgreSQL + PostGIS for structured analytical data.

Suggested domains:

- regions
- departments
- communes
- properties / transactions
- population
- income
- employment
- energy
- transport
- education
- environment

Use dimension-style tables where useful:

dim_commune, dim_department, dim_region, dim_date, dim_property_type.

Use PostGIS for POINT, POLYGON and MULTIPOLYGON geometries and spatial queries.

8. Non-Relational Database

Use MongoDB for:

- raw documents
- reviews
- textual datasets
- NLP outputs
- semi-structured source data

Example document:

```
{
  source: "reviews",
  territory: { commune_code: "33063" },
  text: "...",
  language: "fr",
  sentiment: 0.72,
  topics: ["transport", "noise"]
}
```

9. Data Lake

Implement the standard pipeline:

RAW → BRONZE → SILVER → GOLD

RAW:

Exact source data, preserved unchanged.

BRONZE:

Parsed/converted datasets, preferably Parquet.

SILVER:

Cleaned and standardised data:

- dates
- types
- coordinates
- geographic codes
- missing values
- duplicates
- units

GOLD:

Business-ready analytical datasets:

- housing_market_commune
- housing_market_department
- housing_market_region
- territory_profile
- housing_price_evolution
- climate_risk
- territory_text_insights

10. Data Ingestion

Give every source its own connector rather than building one giant script.

Conceptual interface:

class SourceConnector:

extract()

validate()

normalize()

write_raw()

run()

Record ingestion metadata:

- source
- timestamp
- rows received
- rows processed
- rows rejected
- execution time
- status

11. Data Quality

Before analytical processing, validate:

- schema
- null rates
- duplicates
- date validity
- geographic validity
- numeric ranges

- referential integrity

Examples:

- price > 0
- surface > 0
- valid latitude/longitude
- valid commune code
- valid transaction date

Document rejected records and reasons.

12. Standardisation

Because datasets come from different providers, create canonical fields.

Examples:

- commune_code
- department_code
- region_code
- year / month / date
- latitude / longitude
- price / price_m2
- population
- population_density

Standardise units, encodings, dates, identifiers and categorical values.

13. Apache Spark / Big Data

Use Spark for genuine distributed processing, not merely as a dependency.

Pipeline example:

DVF → Spark → clean transactions → calculate price/m² → aggregate commune → aggregate department → aggregate region.

Use:

- PySpark DataFrames
- Spark SQL
- Parquet
- partitioning
- caching only where justified

Demonstrate measurable distributed transformations and explain the cluster architecture.

14. Housing Analytics

Core indicators:

- median price/m²
- average price/m²
- transaction count
- price evolution
- property-type distribution
- surface distribution

Required geographic levels:

- commune
- department

- region

Create reusable analytical functions rather than embedding calculations in the UI.

15. Temporal Analytics

Build monthly, quarterly and yearly aggregations.

Metrics:

- year-over-year growth
- 3-year evolution
- 5-year evolution
- transaction volume evolution

Keep time dimensions explicit so users can select periods and compare trends.

16. Territorial Profiles

Create a canonical TerritoryProfile combining:

- identity
- housing
- demographics
- economy
- education
- transport
- environment
- energy
- infrastructure
- textual indicators

This becomes the common analytical object consumed by the application.

17. AI / NLP Pipeline

The textual source must be processed, not merely displayed.

Pipeline:

Text → language detection → cleaning → sentence segmentation →
sentiment analysis → topic extraction → entity extraction →
aggregation by territory.

Possible outputs:

- sentiment_score
- topics
- complaint categories
- entities
- keywords

Document model, input, output, preprocessing, evaluation and limitations.

18. Analytics Layer

Create deterministic analytical functions such as:

- calculate_price_evolution()
- calculate_population_growth()
- calculate_housing_density()
- calculate_energy_distribution()

- `calculate_climate_exposure()`
- `calculate_transport_accessibility()`

Pipeline:

`raw data → metric → indicator → visualisation.`

19. Streamlit Application

Suggested application structure:

```

Home
■■■ Market Overview
■■■ Map Explorer
■■■ Territory Explorer
■■■ Housing Market
■■■ Demographics
■■■ Environment & Risk
■■■ Text Intelligence
■■■ Data Explorer

```

Use global filters shared by maps, tables and charts.

20. Map Explorer

Make the map a central feature.

User selects:

1. Geographic level: region / department / commune
2. Indicator: price/m², population, income, energy, climate, employment, etc.
3. Visualisation: choropleth, bubble, heatmap, or other appropriate representation.

Avoid chartjunk and choose representations according to the analytical question.

21. Data Tables

Provide:

- sorting
- filtering
- search
- pagination
- column selection
- CSV export

Allow users to cross-reference indicators at commune, department and region levels.

22. Comparison Mode

Allow users to compare multiple territories.

Example:

Bordeaux vs Toulouse vs Nantes

Compare:

- housing
- population
- economy
- environment
- energy
- infrastructure

Keep comparisons factual and traceable to the underlying datasets.

23. Insights Layer

Separate computation from language generation.

Architecture:

Data → deterministic analytics engine → structured insight → optional LLM explanation.

The LLM should explain computed results rather than calculate or invent statistics.

24. Optional Analytical Assistant

A future assistant can translate natural-language questions into tool calls.

Example:

User asks for a comparison of two cities.

Flow:

question → query/parser → analytics tools → PostgreSQL/Spark → structured results → natural-language explanation.

Keep the data retrieval and calculations deterministic and auditable.

25. Testing

Unit tests:

- price/m²
- geographic mapping
- date normalisation
- missing-value handling

Data tests:

- schema
- null percentages
- duplicates
- referential integrity
- geographic validity

Pipeline tests:

source → raw → bronze → silver → gold

Application tests:

- filters
- maps
- tables
- exports
- navigation

26. Observability & DataOps

Track every pipeline:

- source
- timestamp
- rows received
- rows processed
- rows rejected
- duration
- status

Example:

DVF ingestion

Received: 12,452,932

Valid: 12,112,004

Rejected: 340,928

Duration: 4m32s

Track failures and make pipelines rerunnable.

27. Reproducibility

Record:

- source version
- dataset date
- pipeline version
- Git commit
- processing timestamp

Use metadata such as:

data_version

pipeline_version

processed_at

A result should be traceable back to its source and transformation.

28. Documentation Deliverables

README.md

1. Project overview
2. Architecture
3. Data sources
4. Database schema
5. Data cleaning methodology
6. Spark architecture
7. NLP/AI methodology
8. Analytical methodology
9. Visualisation methodology
10. Installation
11. Local execution
12. Testing
13. Limitations
14. Future improvements

Also include a data dictionary and source catalogue.

29. Implementation Roadmap

PHASE 1 — Foundation

01 Repository

02 Docker

03 PostgreSQL

04 MongoDB

05 Spark

06 CI

PHASE 2 — Data

- 07 Geographic reference data
- 08 DVF
- 09 INSEE
- 10 DPE
- 11 Environment
- 12 Transport
- 13 Education
- 14 Economy
- 15 Textual source

PHASE 3 — Data Engineering

- 16 RAW
- 17 BRONZE
- 18 SILVER
- 19 GOLD
- 20 Data validation
- 21 Standardisation
- 22 Spark transformations
- 23 Aggregations

PHASE 4 — Databases

- 24 PostgreSQL schema
- 25 PostGIS
- 26 MongoDB
- 27 Indexes
- 28 Analytical views

PHASE 5 — Analytics

- 29 Housing KPIs
- 30 Demographic KPIs
- 31 Economic KPIs
- 32 Environmental KPIs
- 33 Geographic analysis
- 34 Temporal analysis

PHASE 6 — AI

- 35 Text preprocessing
- 36 NLP
- 37 Sentiment
- 38 Topic extraction
- 39 Entity extraction
- 40 Evaluation

PHASE 7 — Frontend

- 41 Streamlit shell
- 42 Global filters
- 43 Map
- 44 Tables
- 45 Charts
- 46 Territory explorer
- 47 Comparison
- 48 Text intelligence

PHASE 8 — Production

- 49 Testing

50 Logging
51 Monitoring
52 Docker deployment
53 Documentation
54 Demo dataset
55 Deployment

30. MVP Definition

Do not wait for every source before building.

MVP:

- 5–6 strong sources
- geographic reference model
- PostgreSQL/PostGIS
- Spark
- housing analytical layer
- interactive map
- filterable tables
- Streamlit dashboard

Then incrementally add:

- MongoDB
- NLP
- additional sources
- advanced environmental/economic analytics
- optional analytical assistant

31. Senior-Level Architecture Principle

The project should not become:
collect data → pandas → charts → Streamlit.

The intended engineering story is:

heterogeneous data
→ reproducible ingestion
→ data lake
→ distributed processing
→ polyglot databases
→ analytical layer
→ geospatial + NLP intelligence
→ interactive decision-support platform

This approach directly addresses the HOMEPEDIA requirements while keeping the system modular, testable, reproducible and explainable.